

Mini STAAD implementation and industry user guide

Version 0.3.0 | Prepared 13 September 2026

Repository: [Sonu9721/nextGen_staad](#)

This guide explains what was built, how to install and operate it, how the calculations and result reports work, and how an engineering team could evaluate it for industry workflows. It is intended for the project owner, facade engineers, checking engineers and software integrators. A reader should be able to run a supplied model, understand the reported quantities, reproduce the validation evidence and recognize when a model or feature requires further work.

Mini STAAD is a working linear elastic frame analysis and reporting application. Its strongest present fit is controlled evaluation of facade and casement frame models expressed in the supported STAAD syntax. It is a validation build: it is not a certified design package, a complete STAAD.Pro implementation, or a source of automatic structural approval. The use cases in this guide are proposed applications of its implemented capabilities, not claims of customer adoption or proven industrial deployment.

Release evidence	Current result
Automated tests	234 passed and one licensed OpenSTAAD oracle test skipped
Real HTTP workflow groups	14 passed
Original supplied models	Seven preserved models plus a small instructional cantilever
Models completing structural analysis	Samples 1 through 6
Evaluated reference entries	156 of 164 within unchanged tolerances
Remaining evaluated differences	Eight entries in Samples 1 through 3
Missing reference dataset	Sample 5
Unverified reference entries	40 for the unstable original Sample 7
Live licensed STAAD comparison	Not performed

Passing software tests does not mean every engineering model is correct. The test suite deliberately includes models that must be rejected. Likewise, 156 passing entries are envelope comparisons within stated tolerances, not 156 identical values or a universal accuracy percentage.

Contents

1 Product purpose and development work	3
2 Industry suitability and boundaries	4
3 Installation and first launch	5
4 First analysis using Sample 6	6
5 A complete hand calculation tutorial	7
6 Preparing a reliable structural model	8
7 Supported input commands and units	10
8 Calculation pipeline and numerical checks	11
9 Reading results without mixing definitions	12
10 Facade and casement engineering workflows	14
11 Fin reactions and the Sample 7 investigation	15
12 Other practical use cases	16
13 API integration step by step	17
14 Operating settings and data lifecycle	19
15 Test evidence and reference differences	20
16 Troubleshooting and engineering review	21
17 Adoption process and development roadmap	23
18 Repository map and maintenance	24
19 Glossary and source references	25

1 Product purpose and development work

1.1 The starting system

The supplied project was a Python FastAPI service that accepted a STAAD model, queued a job and obtained results through STAAD.Pro and OpenSTAAD. It already contained useful result envelopes, facade classifications, job status endpoints and process handling. Its original mocked tests checked those integrations; they did not independently establish the accuracy of a structural solver. The NextGen model generator itself was not present in that project.

1.2 The software now delivered

An independent numerical engine now parses supported STAAD text, builds spatial frame stiffness, applies loads, solves displacements and recovers forces. In-house mode runs without a Bentley installation or license. The original job API and classification code remain available, and the optional licensed OpenSTAAD path is retained separately. A browser interface adds model upload, supplied examples, geometry review, progress, cancellation and JSON export.

The solver and interface were developed and checked in stages. The first stage implemented the canonical model, unit conversion, space-frame elements, supports, releases, member loads, rectangular floor loading, combinations and the result adapter. The second stage strengthened independent accuracy checks and added continuous physical envelopes alongside the legacy extraction definitions. Version 0.3 added concentrated member moments, integrated Samples 6 and 7, and improved instability diagnostics and rejected-model review.

Component	Work completed	Practical purpose
Input interpretation	Unit-aware parser and explicit validation	Prevent unsupported or malformed input from being silently analyzed
Numerical engine	Axial, torsional and two-plane bending response	Calculate supported frame behavior independently
Connections	Support restraints and local end-release condensation	Represent the intended load path and connection freedoms
Loading	Joint loads, member forces and couples, selfweight and rectangular panel transfer	Apply the loads found in the supplied models
Recovery	End forces, internal fields, translations, rotations and continuous extrema	Find peaks inside members as well as at nodes
Reporting	Original profile extraction plus full physical response	Preserve integration semantics while exposing broader results
Application	Local browser workspace and asynchronous job service	Make analysis accessible to engineers and upstream software
Evidence	Analytical tests, reference comparisons, HTTP tests and archive checks	Make claims reproducible and limitations visible

1.3 What was preserved

Original sample models and reference outputs are retained. Sample 6 also has a separately normalized JSON reference because its supplied text lost JSON quotes and colons. No numerical reference values were edited to make a comparison pass. The original Sample 7 support and release statements remain unchanged. No artificial restraints were inserted to make an unstable model solve.

The delivered code does not include a visual model editor, a NextGen model generator, a section database or design-code capacity checks. Uploading or editing a supported STD file is the current route to defining a model. Geometry displayed in the interface is undeformed geometry; there is no displacement-animation or bending-diagram viewer in this release.

2 Industry suitability and boundaries

2.1 Where this build can be evaluated

The software can support repeatable analysis and reporting experiments for small and medium facade frames whose idealization fits linear prismatic beam theory. For example, a facade team could use it to examine the effect of changing mullion stiffness, trace a wind load through a rectangular panel, or automate an agreed set of load cases. A checking engineer could inspect whether a support arrangement has a load path before comparing forces and deflections with another solution.

These are controlled pilot uses. Project-specific section properties, loading, connection behavior and acceptance limits must come from the responsible engineering process. The application computes response; it does not establish whether an extrusion, fastener, bracket, glass panel or whole facade has sufficient capacity.

Proposed use	Current capability	Boundary
Facade frame response	Spatial prismatic beams, load combinations and member envelopes	No extrusion capacity or facade-system certification
Casement profile reporting	Five existing profile groups and signed envelopes	Classification must match the model convention
Wind load transfer review	Rectangular two-way tributary loading	Pressure must be supplied; aerodynamic pressures are not generated
Fin reaction import	Global/local concentrated forces and moments	Source reactions and connection eccentricities require independent checking
Automated screening	HTTP jobs and structured JSON	No built-in batch dashboard, optimizer or durable production database
Independent checks	Closed-form benchmarks and reproducible comparisons	Full live STAAD equivalence is not established
Teaching and debugging	Transparent models, source code and failure diagnostics	Educational success does not validate a construction design

2.2 Where it should not be treated as a complete solution

This release does not solve shells or solids, glass-plate stresses, soil interaction, local buckling, contact, nonlinear material response, large-displacement response, P-Delta, dynamics or fatigue. It does not calculate wind pressures from a building code, generate code-required combinations, check connection capacity or produce a compliance certificate. It cannot accept every STAAD command simply because the file extension is .std.

Bentley describes STAAD.Pro as a broader structural analysis and design product covering additional analysis types and design workflows. That product scope is materially wider than this implementation. No benchmark in the present evidence establishes that Mini STAAD is more accurate than STAAD.Pro overall. [Bentley product description](#).

2.3 People and responsibilities

The model author defines geometry, member properties, loads, restraints and releases. The checking engineer confirms the physical load path, verifies assumptions, compares suitable independent results and sets project acceptance criteria. The software integrator preserves job status, units, result definitions and error conditions. The project owner decides whether the evidence is sufficient for a particular pilot or deployment. The software does not replace those roles.

3 Installation and first launch

3.1 Requirements

Use a Windows computer with Python 3.10 or newer available as python. The most recent full validation was performed on Windows with Python 3.14. The first setup requires internet access to install Python dependencies. STAAD.Pro is not required for the default in-house backend. Dependencies are installed into a project-local .venv; they are not included in the downloadable source archive.

The engine uses dense matrices and enforces limits of 1000 nodes and 2000 members. Those are input ceilings, not a performance guarantee. The supplied reference models are much smaller. Available memory, model conditioning, number of cases and recovery workload affect runtime. No measured company-wide throughput or production hardware sizing is claimed.

3.2 Download from GitHub

1. Open the repository linked on the first page.
2. Select **Code** and **Download ZIP**, or clone it using Git.
3. Extract the ZIP to a working folder that you can write to. Do not run the application inside the compressed archive.
4. Open the extracted folder containing start_api.bat, README.md, app and engine.
5. Double-click start_api.bat. The first launch creates .venv and installs missing dependencies.
6. Keep the server window open and visit `http://127.0.0.1:8000` in a browser.
7. Check that the interface reports that the in-house engine is ready.
8. To stop the server, press Ctrl+C in its window.

Git users can use these commands:

```
git clone https://github.com/Sonu9721/nextGen_staad.git
cd nextGen_staad
.\start_api.bat
```

3.3 Manual setup

Use manual setup when you want to inspect dependency installation or diagnose a launcher problem. Run each line in the repository root.

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

The browser API documentation is at <http://127.0.0.1:8000/docs>. A simple health check is <http://127.0.0.1:8000/health>. If port 8000 is already occupied, stop the earlier instance or use another local port and open the corresponding address. Keep a single API process because job records are held in memory.

3.4 Optional configuration

.env.example shows a small set of common settings. If you create a local .env, keep it out of version control. For path settings, prefer explicit process environment variables before startup because runtime directories are configured during application bootstrap. In-house defaults place scratch files, jobs and logs under the repository's runtime directory.

Linux and macOS can use the Python module commands with the appropriate virtual-environment executable. Those systems were not the platform of the final Windows retest, so run the suite and workflow checks on the actual target host before relying on portability.

4 First analysis using Sample 6

4.1 Run the model

1. Start the server and open its home page.
2. In **Explore a supplied model**, select **06 Sample 6 Frame**. The example button selects the casement extraction flow and starts analysis.
3. Watch the progress panel while the model is parsed, assembled, solved and reported. Progress is workflow completion, not an accuracy score.
4. Wait for **Analysis complete**. Do not treat a queued or running job as a finished calculation.
5. Confirm the geometry reports **44 nodes, 61 members and 20 supports**.
6. Confirm that the selected cases are **1, 2, 3 and 4**.
7. Inspect the geometry using Front, 3D, Member IDs, drag-to-rotate and scroll-to-zoom.
8. Read the default physical-response cards, then examine the profile table.
9. Switch **Result definition** to the legacy extraction view to see how the reporting definition changes the values.
10. Select **Export results** to save the complete JSON job response.

4.2 Expected physical values

Quantity	Expected value	Display interpretation
Maximum absolute local MZ	3.198388053 kN-m	Rounded to 3.198 in the card
Maximum absolute local FY	6.843225961 kN	Rounded to 6.843
Maximum absolute local FX	2.281542034 kN	Rounded to 2.282
Absolute resultant displacement	11.868198629 mm	Rounded to 11.868
Maximum member chord-relative resultant	5.729748573 mm	Available in exported physical response

The legacy global displacement is approximately 11.817500 mm. That is a different reporting definition; switching views is not changing the applied loads or rerunning a different structure. All 40 supplied Sample 6 reference entries pass the established comparison tolerances. A different governing member identifier can occur for symmetric or tied results; review the comparison locations instead of assuming that an identifier mismatch is automatically a numerical failure.

4.3 Run your own model

Select or drop a supported STD file, choose its extraction flow, optionally attach the existing generation-request JSON used by your upstream system, and click **Run analysis**. Do not attach a reference result JSON as the profile mapping: they serve different purposes. The optional mapping does not define material stiffness, supports or loads. If an example is being run, clear any mapping that belongs to a different project.

Export the input file and result together with the solver version, chosen flow and engineering assumptions. The service cleans temporary files according to its retention policy; the engineer's saved record should not depend on the temporary job folder remaining present.

5 A complete hand calculation tutorial

5.1 What the example represents

The instructional model `examples/tutorial/cantilever.std` is a four-metre beam along global X, fixed at node 1 and loaded downward at node 2 by 1000 N. It uses $E = 200000000000$ N/m² and local bending inertia $I_Z = 0.00004$ m⁴. No shear areas or bounding dimensions are specified, so this benchmark uses the Euler-Bernoulli limit. It is a mathematical teaching case, not a recommended physical member design.

```
STAAD SPACE
UNIT METER NEWTON
JOINT COORDINATES
1 0 0 0; 2 4 0 0
MEMBER INCIDENCES
1 1 2
DEFINE MATERIAL START
ISOTROPIC TEST
E 200000000000
POISSON 0.3
DENSITY 0
END DEFINE MATERIAL
CONSTANTS
MATERIAL TEST ALL
MEMBER PROPERTY AMERICAN
1 PRIS AX 0.01 IX 0.00001 IY 0.00002 IZ 0.00004
SUPPORTS
```

```

1 FIXED
LOAD 1 TITLE TIP_FORCE
JOINT LOAD
2 FY -1000
LOAD COMB 2 TITLE DOUBLE
1 2
PERFORM ANALYSIS
FINISH

```

5.2 Verify the primary case by hand

1. The support balances the 1000 N downward force with a 1000 N upward reaction.
2. The fixed-end bending-moment magnitude is force times length: $1000 \times 4 = 4000 \text{ N-m}$, or 4 kN-m .
3. The flexural rigidity is E times I_Z : $200000000000 \times 0.00004 = 8000000 \text{ N-m}^2$.
4. The tip-deflection magnitude is $P \times L$ cubed divided by $3 \times E \times I_Z$: $1000 \times 64 / 24000000 = 0.002666667 \text{ m}$, or 2.666667 mm .
5. The tip-rotation magnitude is $P \times L$ squared divided by $2 \times E \times I_Z$: 0.001 rad .
6. Combination 2 applies a factor of two to case 1, so its force, moment, displacement and rotation magnitudes double.

5.3 Run and compare

```

.\.venv\Scripts\python.exe -m engine examples/tutorial/cantilever.std `
--flow standard --primary-only --output tutorial-primary.json

```

Read `physical_response.forces.MZ.value` and `physical_response.displacement.absolute.value`. They should be approximately 4 kN-m and 2.666667 mm . Remove `--primary-only` and run again: the governing values become 8 kN-m and 5.333333 mm because combination 2 is now included. A result twice the hand calculation is therefore correct when that combination governs.

The exported envelopes do not include a full support-reaction table. A developer can access reactions and member rotations through the Python solver result in Section 13. The hand-calculation reaction is stated here as a physical cross-check, not as a claim that a reaction table exists in the interface.

6 Preparing a reliable structural model

6.1 Geometry and connectivity

1. Define an unambiguous global coordinate system and record what X, Y and Z mean for the project.
2. Give every node finite coordinates and a unique positive identifier.
3. Connect each two-node member to the intended start and end nodes. Local x follows that incidence direction.
4. Split members at physical intersections and connect shared nodes. Visually crossing lines do not create a connection automatically.
5. Avoid duplicate overlapping members, zero-length members and unused nodes.
6. Check BETA and the orientation of the section's principal axes before interpreting local moment and shear values.

6.2 Section and material data

Supply positive AX, IX, IY and IZ. AX is cross-sectional area; IY and IZ are bending second moments. The implementation uses IX as the Saint-Venant torsional constant. For an arbitrary section, that torsional constant is not generally the same as the polar second moment. Confirm the property convention with the section supplier or section-property calculation.

Supply Young's modulus E, Poisson ratio and weight density in the active units. Weight density is force per volume, not mass per volume. When converting a mass density to weight density, use the appropriate gravitational acceleration once; do not accidentally apply gravity twice. Supply effective shear areas AY and AZ where available. Do not substitute a hollow section's bounding solid area for its actual structural area.

When AY/AZ are not supplied but YD and ZD are both present, this engine estimates effective shear areas using a rectangular Cowper coefficient multiplied by AX. That is an explicit approximation for arbitrary hollow aluminium extrusions. It should not be presented as a manufacturer-validated extrusion shear area. When dimensions and shear areas are absent, transverse shear deformation is omitted.

6.3 Supports and releases

Support restraints use global directions. PINNED restrains UX, UY and UZ while leaving rotations free. FIXED restrains all six node freedoms. FIXED BUT FY MX MY MZ leaves vertical translation and all rotations free, restraining global X and Z only. With gravity along Y, this is a wind-only support under the user's stated policy.

Member releases use local end directions. START FX MY MZ removes axial-force transfer and both local bending-moment transfers at that member start. FX in a release refers to the member's local axial force; it does not mean global X. For a vertical member, releasing local FX can remove a vertical gravity load path.

Every loaded connected assembly needs a physically valid restraint path. A nearby pinned support is insufficient if an intervening release disconnects the necessary force. The solver can identify instability, but it cannot decide which real-world connection should be changed. Keep original and revised model versions separate and have the load-path decision checked.

6.4 Loads and combinations

Define characteristic or factored loads according to the project methodology and label them clearly. Avoid double-counting selfweight, imported reactions and member gravity loads. Confirm whether a fin-reaction export already includes wind or gravity before adding it to another combination. Linear load combinations superpose primary cases; they do not introduce second-order behavior or replace the engineer's combination rules.

7 Supported input commands and units

Input family	Supported behavior
STAAD SPACE	Spatial frames with six node freedoms
JOINT COORDINATES and MEMBER INCIDENCES	Explicit topology, semicolon records, supported continuations and ranges
UNIT	Active per-statement length and force conversion
Isotropic material	E, POISSON and weight DENSITY
PRIS	Positive AX, IX, IY, IZ and optional AY, AZ, YD, ZD
BETA	Rotation of local section axes about member x
SUPPORTS	PINNED, FIXED and FIXED BUT
MEMBER RELEASE	Local start/end axial, shear, torsional and bending releases, subject to compatibility
JOINT LOAD	Global forces and moments
SELFWEIGHT	Global X, Y or Z direction with a factor
MEMBER LOAD CON	Concentrated force with explicit member station
MEMBER LOAD CMOM	Concentrated moment with explicit member station
MEMBER LOAD UNI	Full or partial uniform force loading
FLOOR LOAD	Full axis-aligned rectangular panels and two-way tributary transfer
Load cases	Primary cases and linear combinations of primary cases
Analysis	Supported PERFORM ANALYSIS and FINISH sequence

7.1 Unit discipline

The canonical solver uses N, m, Pa and N-m. The report uses kN, mm and kN-m. Internal member station distances are metres. A change of UNIT in the STD file affects the following records, so a file may contain multiple legitimate unit systems. Section inertias scale with the fourth power of length, areas with the second power, uniform line loads with inverse length, and pressure with inverse area. A centimetre-versus-metre mistake is therefore much more severe for inertia than for coordinates.

For a concentrated member moment, the magnitude uses force times length. UNIT METER KN followed by 1 CMOM GX 0.02 1.3 represents 0.02 kN-m about global X at 1.3 m from member 1's start. A direction written X, Y or Z is local; GX, GY or GZ is global. The global moment is transformed into the member's axes before its consistent load vector is formed.

7.2 Pressure transfer assumptions

FLOOR LOAD transfers pressure to bordering members without adding a structural plate. The implemented generator requires complete rectangular panels aligned with the global coordinate axes. It traces connected boundaries and creates triangular or trapezoidal member loading. It verifies the total transferred force against pressure times area.

This is a prescribed two-way tributary idealization. It is not a glass-panel plate analysis and does not determine whether a real facade transfers load one-way, two-way or through discrete clips. That modeling decision must be established separately. Bentley's public FLOOR LOAD explanation describes this general pressure-to-member-load concept. [Bentley load transfer explanation](#).

7.3 Unsupported input

Examples include UMOM distributed moments, load offsets, member offsets, section-table lookup, inclined supports, springs, settlements, temperature loads, truss/tension-only members, shells, nonlinear analysis, dynamics and design commands. Nonrectangular or clipped floor panels are outside this generator. Unsupported structural commands produce explicit errors rather than being ignored. The detailed subset is maintained in `docs/SUPPORTED_STAAD_FEATURES.md`.

8 Calculation pipeline and numerical checks

8.1 From text to response

1. **Parse and normalize.** Read topology, properties, restraints, releases and loading using the active units. Reject unsupported commands and nonfinite values.
2. **Build elements.** Each member supplies a 12 by 12 local stiffness matrix for its two six-freedom nodes. The element includes axial response, torsion and bending in both planes.
3. **Orient the model.** Rotate local forces and displacements into the global coordinate system, including BETA.
4. **Apply releases.** Condense both stiffness and equivalent loading at released member-end freedoms. Recover the released end movements after solving.
5. **Generate loads.** Add joint actions, selfweight, concentrated forces/couples and distributed or tributary member loads. Preserve their actual stations.
6. **Assemble and solve.** Assemble the global stiffness matrix, apply support conditions, scale the system and solve the primary load cases.
7. **Recover fields.** Calculate member end forces, in-span force functions, rotations and translations. Check endpoint compatibility.
8. **Combine and report.** Superpose linear combinations, compute physical extrema, apply the legacy reporting definitions and return JSON.

In compact matrix notation the equilibrium problem is $K u = f$: stiffness K relates displacement u to the applied load vector f . A successful numerical solve is meaningful only when the specified restraints and member connectivity represent a stable physical model.

8.2 Beam theory

Timoshenko bending includes both bending curvature and transverse shear flexibility. The Euler-Bernoulli limit is used when shear flexibility is absent. The shear modulus is derived from E and Poisson ratio for an isotropic material. The rectangular Cowper correction is documented in the cited beam-theory literature; its use for a particular extrusion still requires an appropriate engineering assumption. [Kennedy, Hansen and Martins](#).

Concentrated moments are work-conjugate to cross-section rotations. Their load vector uses rotation shape functions, which can differ from displacement slopes when shear deformation is present. Couples introduce jumps in bending moment or torsion. The solver retains those jumps and evaluates both interior limits when finding extrema. At a member endpoint it uses the inward section limit so an external endpoint action cannot create a fictitious span peak.

8.3 Checks that run during analysis

The solver validates dimensions, connectivity, loads and properties. It checks the scaled stiffness eigenvalues for instability or excessive ill-conditioning and verifies finite results. It evaluates backward solution error for each primary case, checks member endpoint compatibility, and independently balances original physical loads against actual support reactions. Combination balances are also reported.

Only a completely disconnected, unloaded rotational freedom may be omitted. A free translation or a coupled structural mechanism is not silently stabilized. The diagnostics include equilibrium residuals, conditioning information, model size and timing. A pure couple has no resultant force, so equilibrium normalization uses dimensionally consistent moment-over-length and force-times-length scales where needed.

8.4 What these checks cannot prove

Numerical equilibrium cannot verify an incorrect wind pressure, a wrong section inertia, a missing bolt, an unintended release or an unsuitable beam idealization. Two programs can agree on the same wrong model. Conversely, two valid reports can differ if one uses sampled legacy section displacement and the other uses a continuous physical displacement definition. Validation must compare both the model assumptions and the exact quantity being reported.

9 Reading results without mixing definitions

9.1 The two reporting views

The default physical response covers all members, includes shear deformation where modeled, and searches continuous piecewise fields for extrema. The legacy extraction view preserves the supplied integration contract and classification logic. Its shear envelopes use member ends, some axial envelopes use classified inner mullions, and section displacement follows the retained legacy recovery and station sampling. The profile table always retains legacy definitions even when the cards show physical values.

Bentley documents a difference between joint displacement recovery and intermediate section recovery when shear is included. That explains why comparing differently defined displacement outputs can be misleading. It does not establish that every remaining discrepancy in this project is caused by that issue. [Bentley section-displacement explanation](#).

JSON section	Meaning
analysis	Selected cases, backend, solver version, validation status and diagnostics
units	Output units and the source units used by the adapter
properties	Legacy global and applicable grouped extraction values
profiles	Applicable fully unitized profile classification metadata
casement	Signed casement group envelopes and member lists
physical_response.forces	All-member continuous envelopes for FX, FY, FZ, MX, MY and MZ

JSON section	Meaning
physical_response.displacement	Absolute and member chord-relative resultant peaks

9.2 Absolute and relative movement

Absolute resultant displacement measures the magnitude of a point's global translation. Chord-relative displacement measures departure from the straight line joining the displaced member endpoints. A member can have small bending relative to its chord while its entire supporting frame translates substantially. Serviceability assessment must use the quantity associated with the actual project criterion, not whichever number is smaller.

The console's maximum-displacement card in physical mode uses the absolute resultant. Chord-relative physical values are in the JSON. Casement profile displacement follows its own retained chord-relative and selected-node rules; it should not be substituted blindly for a whole-structure drift limit.

9.3 Axes, signs and locations

FX is local axial force, FY and FZ are local shears, MX is torsion, and MY/MZ are local bending moments. A facade member rotated with BETA may have local directions very different from global X/Y/Z. The term major in the present default reporting refers to the configured MZ or FY extraction convention; it does not automatically prove which physical extrusion axis is strongest.

Physical force records include an absolute value and a signed_value. Casement min/max records retain signs. A member-end force is an action at that end and need not have the same sign convention as an interior cut from the other end. Always preserve the direction, station, case and record type when transferring a value into another calculation.

A station reported in metres is distance along the member. In legacy member-end records, station 0 or 1 can be an end identifier rather than a length. Some supplied reference stations are inches. Use an explicit station_unit when present and the applicable source-unit metadata otherwise. Never compare raw station numbers without resolving this convention.

9.4 Limits of an envelope report

An envelope is a maximum or minimum over selected cases and locations. Maximum axial force, shear and bending may occur in different cases; combining them as if they occurred simultaneously can create an invalid design action set. Obtain concurrent case-specific actions for downstream interaction checks. This API export is not a full per-member/per-station database, although the Python solver object exposes more detailed fields for custom tooling.

An empty group means no members matched that classifier. It is not proof that the force is zero or that the corresponding component is safe. Review the member lists and upstream generation metadata when a group is unexpectedly empty.

10 Facade and casement engineering workflows

10.1 Facade frame response study

Purpose: study the response of a supported mullion/transom frame under agreed gravity and wind loading before a separate capacity and connection review.

1. Obtain the actual grid, span lengths, section properties, supports and connection freedoms.
2. Confirm the global wind direction and the meaning of every BETA angle.
3. Establish whether panel loads transfer one-way, two-way or through point attachments. Use FLOOR LOAD only when its supported transfer assumption fits the model.
4. Define primary gravity and wind cases and the combinations required by the project's engineering basis.
5. Run a supported model, inspect the geometry and confirm classification lists before reading profile envelopes.
6. Review continuous physical force and displacement maxima, then identify the governing case and member.
7. Check relevant cases independently and pass concurrent design actions to the responsible section and connection checks.
8. Save the exact input, solver version, output, comparison evidence and reviewer decision.

Useful output: traceable member-force and displacement envelopes for the specified linear model.

Additional work: aluminium or steel member capacity, local effects, bracket/anchor resistance, glass design, movement compatibility and project compliance.

10.2 Casement profile report

Purpose: organize supported frame response into the existing interlock, central meeting, fixed mullion, horizontal and outer profile groups.

1. Begin with a model that follows the supplied casement conventions; Sample 4 or Sample 6 is a practical starting point.
2. Select the casement extraction flow and attach generation metadata only if it belongs to that exact model.
3. Confirm member counts and IDs for each profile group. Classifications use existing property and geometry conventions, not a general CAD understanding of every window system.
4. Read signed minima and maxima for bending, shear and axial force, retaining their load cases and stations.
5. Review profile deflection under its documented chord-relative and selected-node definition.
6. Use the physical global results to detect response that may fall outside a particular group summary.
7. Resolve empty or unexpected groups before issuing a profile report.

The program does not simulate opening/closing hardware, contact between sashes, seals, friction, latch behavior or glass flexibility. Those mechanisms require a separate appropriate model or check.

10.3 Comparing section alternatives

Purpose: compare candidate section stiffnesses without losing track of the input assumptions.

1. Duplicate a verified input into clearly named alternatives.
2. Change only the intended section properties and record their source. Preserve dimensions, boundary conditions and loads unless the study explicitly changes them.
3. Update weight density times area effects consistently; changing area changes selfweight when SELFWEIGHT is active.

4. Run the alternatives with the same cases, flow and result definition.
5. Compare deflection and concurrent force demands against the engineer's criteria.
6. Check strength, stability, connections, manufacturability and cost separately.

A stiffer member can redistribute load in an indeterminate frame. Lower displacement does not by itself mean lower force in every connection. Automated optimization and a built-in product catalog are future possibilities, not current features.

11 Fin reactions and the Sample 7 investigation

11.1 Importing a known reaction

Fin or secondary-component reactions can be applied as forces and couples when the source analysis, axes, locations and connection behavior are known. A force acting at an eccentric attachment produces a moment equal to the vector cross product of eccentricity with force. Its sign depends on those vectors; an unsigned magnitude alone does not determine the correct rotational direction.

1. Record the source component, source load case and reaction coordinate system.
2. Determine the equal-and-opposite action transferred into the receiving frame.
3. Transform that action into the receiving frame's global axes.
4. Locate the attachment along the receiving member from its actual start node.
5. Apply the force using CON or a joint load, and its associated couple using CMOM or a joint moment.
6. Check that the same load has not also been included through panel pressure or another reaction export.
7. Define combinations that respect what the imported reaction already contains.
8. Verify that the receiving frame has a physical path to the supports for each transferred action.

11.2 What Sample 7 contains

The preserved model has 48 nodes, 68 members, primary cases 1, 2 and 5, and combinations 3, 4 and 6. Its 48 concentrated moment records are now supported by the engine. They include global bending/torsional actions associated with fin reactions. The parser accepts this loading; the analysis subsequently fails a stability check because of the original support/release arrangement.

The user's stated policy is retained: pinned supports resist gravity and wind; other supports resist wind only. The original file already implements that policy. Its pinned nodes are 7, 11, 19, 23, 31, 35, 43 and 47. The other twelve supports restrain global X and Z while allowing global Y translation.

11.3 The unresolved gravity load path

Nodes 1 through 5, 13 through 17, 25 through 29 and 37 through 41 form a lower assembly. No node in that assembly has a vertical support. Local axial releases at the starts of members 5, 16, 27 and 38 disconnect its vertical force path to the pinned supports above.

A direct verification moves all 20 lower nodes together vertically. The resulting normalized stiffness residual is about 8.66e-19, effectively a rigid mode, while gravity performs nonzero virtual work corresponding to 11994.59676816 N per metre of that movement. The original model therefore cannot have a valid linear static gravity solution.

The application reports SINGULAR_MATRIX with the affected nodes, keeps the original geometry available and returns no analysis result. The supplied reference file is attached as evidence but is not substituted for a computed solution. Its 40 values remain unverified.

11.4 How the issue should be resolved

1. Inspect the actual lower-to-upper connection details.
2. Decide whether the starts of members 5, 16, 27 and 38 physically transmit axial force.
3. If they do, an engineer can assess changing those releases from FX MY MZ to MY MZ in a separate revised model.
4. If they do not, identify the actual missing gravity support or another valid load path.
5. Have the revised connection/support assumptions checked before accepting a solution.
6. Rerun stability, equilibrium, displacement and independent comparison checks using the revised model.

No such physical connection change has been authorized or applied in the delivered original. The earlier support-policy confirmation alone does not establish axial continuity at a released joint.

12 Other practical use cases

12.1 Upstream NextGen integration

A NextGen service can generate a supported STD file, submit it to the job API and use the returned structured results in its own reporting workflow. The advantage is an explicit job boundary with progress, failure and result states. To evaluate this use, freeze an agreed model convention, verify each generated command is supported, validate profile classifications and compare representative models before accepting a production integration. The generator itself must be maintained outside this repository.

12.2 Regression checking after a model generator change

An engineering software team can keep a representative model corpus and rerun it after changing geometry generation, load allocation or section mappings. Compare input provenance, member counts, cases, classifications, equilibrium and values rather than only checking that the program exits successfully. A failed mechanism test may reveal a newly omitted support; a shifted profile group may reveal a property-order assumption. This workflow is a proposed use of the test and API tooling, not a built-in continuous-monitoring service.

12.3 Teaching structural behavior

Use the cantilever tutorial to connect force, stiffness and displacement, then add a known tip moment or vary a load factor. Explain why doubling a linear load doubles the response, and why changing section stiffness affects deflection. Next compare a single member carrying an interior couple with a subdivided beam carrying an equivalent nodal moment. The independent tests use that equivalence to check implementation behavior.

12.4 Support and connection review

A checker can inspect whether a proposed idealization carries gravity and wind into the intended supports. Begin with an explicit load path and verify the support axes and member-local releases. Run the model, investigate a mechanism diagnostic and compare the response of separately justified alternatives. The software identifies a numerical consequence of the specified model; it does not choose a real connection detail.

12.5 Repetitive report preparation

An internal reporting tool can consume the exported JSON, preserve units and governing locations, and place selected quantities in an engineering report template. A useful record includes the source STD, model revision, solver version, extraction flow, all selected cases and reviewer decision. There is no built-in Word/PDF structural report generator in the analysis application; the present document is a project guide, and a production report generator would be a separate integration.

12.6 Lightweight frame studies outside facades

The generic engine can represent some other prismatic spatial beam frames within the same restrictions. Standard extraction is more appropriate than assuming facade-specific groups. Before extending use to another structural family, establish independent benchmark coverage for its geometry, restraints, loading, slenderness and member theory. Do not extrapolate the current facade sample comparisons to bridges, towers, offshore structures or other complex systems.

13 API integration step by step

13.1 Endpoints and states

Request	Purpose	Typical successful response
GET /health	Server health	HTTP 200
GET /health?check=staad	Backend readiness	HTTP 200 when healthy
GET /health?check=license	License status for selected backend	In-house mode requires no Bentley license
POST /jobs	Submit STD and extraction settings	HTTP 202 and a job ID
GET /jobs/{job_id}	Poll progress and status	Status object
GET /jobs/{job_id}/result	Retrieve completed response	Job wrapper containing result when succeeded
POST /jobs/{job_id}/abort	Request cancellation	Abort response
GET /jobs/{job_id}/model	Undeformed review geometry	Nodes, members and supports when available

The terminal states are succeeded, failed and canceled. Queued and running are intermediate. A failed structural result can return HTTP 500 from the result route; the status route supplies the structured error. A polling client must not retry every HTTP 500 as if it were a temporary network failure. Unsupported input and structural instability are nonretryable analysis outcomes in the in-house path.

13.2 Submit a model

The following commands assume the server is running locally and the current folder is the repository root. Use `curl.exe` explicitly in PowerShell.

```
curl.exe -X POST http://127.0.0.1:8000/jobs `
  -F "std_file=@examples/sample6/sample_6.std" `
  -F "extraction_flow=casement"
```

The response contains a `job_id`. Replace `JOB_ID` in the following requests with that returned identifier.

```
curl.exe http://127.0.0.1:8000/jobs/JOB_ID
curl.exe http://127.0.0.1:8000/jobs/JOB_ID/result -o sample6-job-result.json
```

Poll the status until it reaches a terminal state before consuming the result as a completed analysis.

Cancellation uses `curl.exe -X POST http://127.0.0.1:8000/jobs/JOB_ID/abort`. A client should retain the final status and error message rather than showing a blank report after cancellation or failure.

13.3 Integration sequence

1. Validate the upstream model convention and select standard, fully_unitized or casement explicitly.
2. Submit the STD using the `std_file` multipart field. Send `generation_request` only when the existing upstream mapping requires it.
3. Store the returned job ID with the upstream project and revision.
4. Poll status at a reasonable interval, checking for failed and canceled as well as succeeded.
5. Retrieve the completed job wrapper and inspect its backend, version, units and selected cases.
6. Decide explicitly whether the downstream report uses `physical_response` or the legacy fields.
7. Preserve governing locations and concurrent case information required by the downstream checks.
8. Save the input and output outside temporary job storage and attach reviewer decisions.

13.4 Command line and Python access

The CLI writes the result payload directly, whereas the HTTP endpoint and browser export provide a job wrapper around that payload. Account for that difference when reading JSON.

```
.\.venv\Scripts\python.exe -m engine examples/sample6/sample_6.std `
--flow casement --output result.json
```

For developer tools needing case-specific reactions or rotations:

```
from pathlib import Path
from engine.parser import parse_std
from engine.solver import solve

solution = solve(parse_std(Path("examples/tutorial/cantilever.std")))
case = solution.cases[1]
node_index = solution.node_indices[1]
reaction = case.reactions[6 * node_index:6 * node_index + 6]
member = case.members[1]
tip_rotation = member.local_rotation(member.element.length)
```

Raw solver reactions use N and N-m, translations use metres, and rotations use radians. They must not be interpreted as already converted report units. Node reactions are global; member internal quantities are local. This Python interface is suitable for controlled custom tooling and should be version-pinned by an integrator.

14 Operating settings and data lifecycle

Setting	Default or meaning	Operational implication
ANALYSIS_BACKEND	inhouse	Independent numerical path is the default
STAAD_WORKER_COUNT	1	Keep one worker for the documented setup
ALLOW_PARALLEL_STAAD	false	Worker execution remains serialized by default
STAAD_API_TIMEOUT_SECONDS	180 seconds	Default job timeout; does not guarantee completion of every model
EXTRACTOR_INCLUDE_COMBINATIONS	true	Combinations are included unless explicitly excluded
STAAD_API_MAX_UPLOAD_BYTES	52428800 bytes	Input upload ceiling; structural model limits are separate
KEEP_JOB_FOLDER_ON_ERROR	false	Failed-job files are normally cleaned
JOB_RETENTION_HOURS	6	Stale job-folder cleanup interval basis
COMPLETED_JOB_TTL_SECONDS	3600	Finished in-memory job metadata can expire
TEMP_DIR, STAAD_JOBS_DIR, LOGS_DIR	Process path overrides	Set before startup when custom storage is needed

14.1 Persistence and cleanup

Jobs are held in memory. Restarting the service loses its job registry, and running multiple independent API processes does not create a shared registry. Temporary files and logs are not the permanent engineering record. Export the input, result and evidence needed for review. The Sample 7 instability path retains undeformed geometry in its in-memory record even after failed-job file cleanup; that geometry still expires with the record.

In-house jobs run in owned spawned worker processes with cancellation and timeout handling. They do not launch or terminate Bentley applications. The retained OpenSTAAD backend has different dependencies and legacy process-management behavior, so test it separately on an appropriate licensed Windows host. Its presence in the repository is not evidence of a successful live oracle run.

14.2 Local use and future hosting

The launcher binds to 127.0.0.1 for local use. The current service has no built-in user authentication, tenant separation or durable database. Publishing the source on GitHub does not host the analysis application or make it a public engineering service.

Before a shared service is deployed, an implementation plan should cover authentication, encrypted transport, durable job and artifact storage, access controls for client models, resource limits, job isolation, backups, monitoring and a verified recovery process. Those are future operational requirements, not implemented product features. Confidential client models should follow the organization's approved data-handling process.

14.3 Dependency control

requirements.txt lists runtime dependencies. requirements-dev.txt adds test dependencies. requirements-openstaad.txt is for the optional licensed path. requirements-tested.txt records the retested environment. Package versions and operating systems can affect numerical and API behavior; reproduce the checks after dependency changes. Do not assume the launcher upgrades already installed packages or validates their latest versions on every start.

15 Test evidence and reference differences

15.1 Layers of evidence

The 234 passing tests include independent mathematical checks, invariance checks, parser failures, input validation, solver recovery, profile contracts and application behavior. One licensed oracle test is skipped because no live licensed comparison was enabled. The 14 real HTTP groups exercise actual server workflows, including rejection of Sample 7. They are separate from the pytest count and should not be added as if every group were a numerical benchmark.

Closed-form beams check axial response, torsion, bending, reactions and deflection. Virtual-work references independently integrate loads. Subdivision checks compare a beam against an equivalent split model. Other tests cover local/global axes, releases, shear flexibility, linear superposition, material/load scaling, survey-coordinate invariance and continuous extrema. Concentrated moments add 43 focused cases; normalization adds eight checks.

15.2 Supplied model comparisons

Sample	Members	Passed values	Differences	Reference status
1	57	26	2	Compared
2	105	24	4	Compared
3	54	26	2	Compared
4	57	40	0	Compared
5	70	-	-	No supplied output
6	61	40	0	Compared using separately normalized reference
7	68	-	-	Unstable original model; 40 entries unverified

The tolerance is the larger of 0.1 percent of the reference magnitude and an absolute allowance of 0.01 kN, 0.01 kN-m or 0.05 mm for the applicable quantity. These are the project's established comparison thresholds, not code-prescribed design limits. All comparison entries and governing-location assessments are preserved in validation/comparison.json.

15.3 The eight evaluated differences

Underlying quantity	Reference	In-house legacy value	Difference
Sample 1 displacement in mm	51.836027	51.899914	0.063887
Sample 2 MZ envelope in kN-m	6.788116	6.804171	0.016055
Sample 2 displacement in mm	97.399002	97.527675	0.128673
Sample 3 displacement in mm	71.384957	71.488369	0.103412

Each underlying difference appears in both global and mullion reporting, producing eight failed entries rather than eight unrelated physical discrepancies. For the Sample 2 moment, evaluating the solver at the supplied reference station reproduces the reported reference value while the continuous extremum occurs between reference stations. The displacement differences remain documented and are not corrected by fitting a factor to the sample outputs.

15.4 Reproduce the checks

```
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe scripts/validate_samples.py
.\.venv\Scripts\python.exe scripts/retest_e2e.py
```

The reference validator's `--strict` option intentionally exits with failure while differences, unavailable references or rejected models remain. That is a release-evidence signal, not a reason to loosen tolerances. HTTP workflow success for Sample 7 means it was correctly rejected with a useful diagnostic, not that its structural analysis succeeded.

15.5 What additional validation is needed

Obtain complete member, node and support results for agreed models from a licensed reference run or another appropriate independent oracle. Verify section shear properties and load-transfer assumptions for actual extrusion and connection families. Add representative non-coplanar frames, unusual releases, mixed units and difficult conditioning cases. Review the numerical approach and implementation independently. Certification, code compliance, blanket 100 percent accuracy and superiority to STAAD.Pro have not been established.

16 Troubleshooting and engineering review

16.1 Startup and operating problems

The launcher closes or Python is missing. Open a terminal in the project folder, check that Python 3.10 or newer is available, and run the manual setup commands so the error remains visible. The first dependency installation needs internet access. Do not assume a partial `.venv` installation is complete.

The browser cannot connect. Confirm the server window is still running and that the browser address uses the same host and port. If another application owns the port, use a different local port. A public GitHub repository URL is a source-code location, not the running analysis service.

A job remains running or times out. Inspect its progress and logs. Consider model size, complexity and conditioning. Raising the timeout can allow more computation but does not correct an unstable model. Use cancellation when the run is no longer needed; save valid earlier results outside temporary folders.

An old job ID returns missing. The server may have restarted or the completed record may have expired. Use the saved input and rerun the analysis. Durable job persistence is not implemented.

16.2 Input and numerical errors

Symptom	Likely investigation	Appropriate response
Unsupported command	Input exceeds the documented subset	Model the supported case explicitly or use another qualified solver
Invalid section or material	Missing, nonpositive or nonfinite properties	Check source properties and active units
Zero-length or invalid member	Duplicate coordinates or wrong incidence	Correct the geometry and connectivity
FLOOR LOAD failure	Incomplete, nonrectangular or unconnected panel boundary	Check the intended transfer model and panel topology
SINGULAR_MATRIX	Mechanism or excessive ill-conditioning	Inspect affected freedoms, restraints and releases
MECHANISM_DETECTED	Unrestrained or loaded zero-stiffness freedom or incompatible releases	Establish a real load path before rerunning
Numerical or recovery failure	Equilibrium, compatibility or finite-range issue	Preserve the input and diagnose; do not replace missing results with zero
Empty profile group	Classification convention does not match the model	Inspect member lists, property groups and mapping metadata

16.3 A different number from another report

1. Compare the exact input file and revision, including all UNIT changes.
2. Match material, section, shear areas and torsion properties.
3. Match supports, releases, BETA angles and member start/end directions.
4. Match load cases, factors and any imported reaction assumptions.
5. Compare the same axis, sign convention, station unit and result definition.
6. Evaluate the same member and station before comparing envelope maxima.
7. Investigate remaining differences and keep them visible in the report.

Do not select a more favorable result definition, suppress a mechanism or increase comparison tolerances merely to obtain a pass. An engineering conclusion should identify which model and checks justify the accepted result.

17 Adoption process and development roadmap

17.1 A practical pilot process

1. **Define the pilot family.** Choose a narrow class of facade or casement frames and list the allowed geometry, section, connection and load conventions.
2. **Agree the engineering basis.** Establish the source of loads, effective properties, serviceability definitions and independent checks.
3. **Build a benchmark set.** Include normal cases, extreme supported cases and invalid models that should fail. Keep original input and reference provenance.
4. **Resolve gaps.** Review the eight current differences, obtain missing reference coverage and clarify the original Sample 7 load path before using it as a valid benchmark.
5. **Run in parallel with the established process.** Compare the same quantities without replacing the organization's existing approval workflow prematurely.
6. **Review evidence independently.** Decide which limited uses have adequate numerical and engineering support.
7. **Control releases.** Pin code and dependencies, rerun the evidence set after changes and record reviewer acceptance.

17.2 Measures of a useful pilot

Track how many real project models fall entirely within the supported subset, how many require correction, which reference quantities remain outside tolerance, and whether engineers can trace each result to its source model and case. Measure actual runtime and reporting effort on the target hardware. No saving percentage, adoption rate, cost reduction or production throughput has been measured for this build.

17.3 Prioritized future work

First, close validation gaps. Obtain full licensed reference databases, confirm extrusion shear areas, expand independent coverage and complete the Sample 7 connection review. Document which model families are qualified by that evidence.

Next, improve engineering usability. Potential additions include full case-specific reaction and member-result exports, force diagrams, deformed-shape visualization, explicit local-axis graphics, more transparent profile mapping and a dedicated model editor. These are proposed features, not hidden capabilities in the current interface.

Then, prepare operational deployment. Durable jobs and artifacts, authentication, resource isolation, audit records, backups and monitoring would support a shared internal service. A sparse numerical strategy could be evaluated if larger model sizes become an actual requirement.

Only with separate development and validation, expand analysis scope. Springs, offsets, more general loads, nonlinear effects, buckling, dynamics, shells or design-code checks require new formulations and independent evidence. They should not be added as a checkbox that simply passes unsupported commands through the current solver.

18 Repository map and maintenance

18.1 Where to find the implementation

Path	Responsibility
engine/parser.py, model.py, units.py	Input interpretation, canonical data and conversion
engine/element.py	Local axes, stiffness, consistent loads and release handling
engine/loads.py	Member loads and rectangular tributary generation
engine/solver.py	Assembly, solution, combinations and diagnostics
engine/recovery.py, physical.py	Internal fields, translations, rotations and physical extrema
engine/adaptor.py	Bridge to the retained classification and envelope contract
engine/comparison.py	Reference comparison and location assessment
engine/worker.py	Bounded worker entry point
app/main.py, job_manager.py	HTTP endpoints, queue and lifecycle
app/services/inhouse_runner.py	Owned in-house process and cancellation/timeout handling
app/console.py, app/web	Local review interface and geometry endpoints
staad-max-extractor	Original OpenSTAAD integration and profile reporting
examples	Original sample models, references, provenance and tutorial
tests	Numerical, parser, API and integration checks
validation	Recorded outputs, comparisons and retest evidence
scripts	Reproducible validation, normalization and packaging utilities
docs	Architecture, supported syntax, limitations and user documentation

18.2 Maintenance workflow

1. Create a branch for an intended change and keep the model/input corpus under review.
2. Write a regression check for a meaningful numerical or behavior defect before changing its implementation.
3. Run the relevant checks, then the full required suite for a numerical or integration change.
4. Run reference comparisons without changing expected values merely to accommodate the implementation.
5. Run real HTTP workflows when endpoint, lifecycle or result behavior changes.
6. Update supported features, limitations, version metadata and evidence together.
7. Review the change and publish a traceable release with its exact model and dependency assumptions.

The test evidence shipped with v0.3.0 records the retest performed before this documentation publication. New tutorial and publication checks are recorded separately; their existence does not retroactively change the 234-test count. Archived v0.2 reports are historical and should not be mistaken for the latest scope.

18.3 Publication contents

The repository includes application and engine code, the retained extractor, tests, examples, reference evidence and documentation. Virtual environments, runtime jobs/logs, editor caches, private local environment files and transient rendering files are excluded. Publishing the source does not publish a running service. No new open-source license or third-party permission is inferred merely from making the repository public; downstream redistribution should follow the applicable ownership and dependency licenses.

18.4 Rebuilding the release and guide

After successful verification, `scripts/package_build.py` creates the portable source archive using the recorded test and HTTP evidence. The guide's editable source is this Markdown file.

`scripts/build_industry_guide.py` builds its PDF with ReportLab; install `requirements-docs.txt` for that separate documentation task. Documentation dependencies are not needed to run the structural analysis application.

19 Glossary and source references

19.1 Glossary

Term	Meaning in this guide
Degree of freedom	A node translation or rotation that may be free or restrained
Stiffness	Relationship between a displacement and the resulting restoring action
Support	Connection between the model and its external restraint system
Release	Removal of a specified member-end force or moment transfer
Mechanism	Motion without sufficient restoring stiffness
Load path	Physical route by which an action reaches its resisting supports
Primary case	An explicitly applied loading scenario before combination
Combination	Linear sum of primary cases multiplied by factors
Envelope	Extremum over a specified set of cases, members or stations
Resultant displacement	Magnitude of the three translation components
Chord-relative displacement	Movement relative to the line joining displaced member endpoints
PRIS	Explicit prismatic section property definition in the supported STD syntax
BETA	Rotation of local section axes about the member's incidence axis
Timoshenko beam	Beam formulation including transverse shear flexibility
OpenSTAAD	Bentley interface used by the retained licensed extraction path
Oracle	Independent reference solution used for validation

19.2 Project evidence

Use docs/BUILD_VERIFICATION.md for the current retest summary, docs/VALIDATION_RESULTS.md and validation/comparison.json for every reference entry, docs/SAMPLE_6_7_VALIDATION.md for provenance and the instability investigation, and docs/KNOWN_LIMITATIONS.md for unresolved scope. The implementation is the authority for actual current software behavior; this manual explains that behavior and its practical implications.

19.3 Public technical sources

The following primary sources were checked while preparing the guide. Their descriptions are background references, not certification of this implementation.

1. [Bentley STAAD product description](#) describes the broader commercial analysis and design product scope.
2. [Bentley pressure transfer to members](#) explains the general FLOOR LOAD pressure-to-triangular/trapezoidal-member-load approach.
3. [Bentley joint and section displacement distinction](#) explains the role of shear deformation in the two reporting methods.
4. [Kennedy, Hansen and Martins on Timoshenko beam theory](#) discusses shear correction, including Cowper's rectangular isotropic coefficient.

The core release conclusions remain specific: supported linear frame analysis is implemented and tested; all Sample 6 reference entries pass within tolerance; eight evaluated earlier entries differ; Sample 5 lacks a reference; and the original Sample 7 requires a confirmed physical gravity load path before it can yield a valid analysis.